```
//*Global Variables*
  //Clock
  var seconds = 0; //Cycles 0 - 59, for time

  //Information about blob
  var name = ""; //stores name of blob
  var currentEmotion = "Happy"; //stores current emotion of blob

  //stores visual state of blob
  var emotionsList = ["Happy", "Playful", "Bored", "Sleepy", "Sad", "Angry", "Hungry"]; //list of
valid emotions
  var emotionsFaces = ["assets/HappyEmotion.png", "assets/PlayfulEmotion.png",
"assets/BoredEmotion.png", "assets/SleepyEmotion.png", "assets/SadEmotion.png",
"assets/AngryEmotion.png", "assets/HungryEmotion.png"]; //pngs tied to emotions
  var costumeTypes = ["Normal", "Pumpkin", "Santa", "Superhero"]; //list of valid costumes
  var costumeImages = ["assets/NormalCostume.png", "assets/PumpkinCostume.png",
"assets/SantaCostume.png", "assets/SuperheroCostume.png"]; //pngs tied to costumes
  var currentCostume = 0;

  //stores internal state of blob
  var maxStatValue = 100; //Global maximum stat value
  var minStatValue = 0; //Global minimum stat value
  var currentHunger = 50; //current hunger (Hungry if below 20) Priority: High
  var currentEnergy = 70; //current energy (Bored if below 40, Sleepy if below 10) Priority: Medium
  var currentHappiness = 60; //current happiness (Playful 100, Happy 75, Sad 50, Angry 25) Priority:
Low

////////////////////////////////////////////////
//*Global Functions*
  // Starts a clock for the blob
  function startClock() {
  //converts milliseconds ran to seconds
  timedLoop(1000, function() { //every 1000 milliseconds...
    seconds++; //...convert to 1 second
    if (seconds > 59) //if seconds exceeds 59
    {
      seconds = 0; //set seconds back to 0
      decreaseInternalState(); //executes internal state decrease
    }
    setText("petScreenClock", "Clock: " + seconds);
  });
  }

  // Decreases internal state of blob by a random amount
  function decreaseInternalState() {
    currentHunger -= randomNumber(10, 20);
    currentEnergy += randomNumber(15, 30);
    currentHappiness -= randomNumber(10, 25);

    currentHunger = stateBounds(currentHunger);
    currentEnergy = stateBounds(currentEnergy);
    currentHappiness = stateBounds(currentHappiness);
    updateStatsAndEmotions();
    updateButtons();
  }

  // Updates the stat text and the face of the blob
  function updateStatsAndEmotions() {
    //Updates Stats
    setText("petScreenHungerStat", "Hunger: " + currentHunger + "%");
    setText("petScreenEnergyStat", "Energy: " + currentEnergy + "%");
    setText("petScreenHappinessStat", "Happiness: " + currentHappiness + "%");

    //Updates Emotions
    currentEmotion = "Happy";

    if (currentEnergy < 10)
    {
      currentEmotion = "Sleepy";
```

```
  }
  else if (currentEnergy < 40)
  {
    currentEmotion = "Bored";
  }
  else if (currentHunger < 20)
  {
    currentEmotion = "Hungry";
  }
  else if (currentHappiness > 75)
  {
    currentEmotion = "Playful";
  }
  else if (currentHappiness > 50)
  {
    currentEmotion = "Happy";
  }
  else if (currentHappiness > 25)
  {
    currentEmotion = "Sad";
  }
  else
  {
    currentEmotion = "Angry";
  }

  setImageURL("petScreenEmotion", findPNG(emotionsList, emotionsFaces, currentEmotion));
  setText("petScreenName", "Hi! My name is "+ name + "! I am " + currentEmotion + ".");
}

// updates the color of the buttons based on energy
function updateButtons() {
  if (currentEnergy >= 10)
  {
    setProperty("petScreenFeedButton", "background-color", rgb(199, 199, 199));
    setProperty("petScreenPlayButton", "background-color", rgb(199, 199, 199));
  }
  else
  {
    setProperty("petScreenFeedButton", "background-color", rgb(92, 92, 92));
    setProperty("petScreenPlayButton", "background-color", rgb(92, 92, 92));
  }
}

// Ensures internal state does not leave the bounds set by the global stat values
// state {int} - internal state #
// return {int} - proper value
function stateBounds(state) {
  if (state > maxStatValue)
  {
    return maxStatValue;
  }
  if (state < minStatValue)
  {
    return minStatValue;
  }
  return state;
}

// Finds an image from an array of images using the index of the states array
// states {array} - states array
// png {array} - images array
// request {String} - requested image from states array that corrisponds with the images array
// return {img} - selected
function findPNG(states, png, request) {
  for (var i = 0; i < states.length; i++)
  {
    if (states[i] == request)
    {
      return png[i];
```

```
    }
  }
  return null;
}

// Cycles all available costumes and applys it to the blob
function cycleCostumes() {
  currentCostume++;
  if (currentCostume >= costumeTypes.length)
  {
    currentCostume = 0;
  }
  setImageURL("petScreenCostume", findPNG(costumeTypes, costumeImages,
costumeTypes[currentCostume]));
}

// Feeds blob if blob has at least 10% energy
function feedBlob() {
  if (currentEnergy >= 10)
  {
    currentHunger += randomNumber(0, 10);
    currentHunger = stateBounds(currentHunger);
    playSound("assets/Feed.mp3", false);
    updateStatsAndEmotions();
    updateButtons();
  }
}

// Entertains blob if blob has at least 10% energy
function entertainBlob() {
  if (currentEnergy >= 10)
  {
    currentEnergy -= randomNumber(0, 10);
    currentHappiness += randomNumber(0, 10);
    currentEnergy = stateBounds(currentEnergy);
    currentHappiness = stateBounds(currentHappiness);
    playSound("assets/Play.mp3", false);
    updateStatsAndEmotions();
    updateButtons();
  }

}

////////////////////////////////////////////////////
//*startScreen*
playSound("assets/Start.mp3", false);
setTimeout(function() {
  playSound("assets/Blobs-Wander-The-World-Loop.mp3", true);
}, 3000);

onEvent("startScreenStartButton", "click", function( ) {
  if (getText("startScreenNameInput") != "")
  {
    name = getText("startScreenNameInput");
    setText("petScreenName", "Hi! My name is "+ name + "! I am " + currentEmotion + ".");
    setText("rulesScreenRulesText", "This is your new sentient blob, " + name + "! You can play and
feed this little alien being, but make sure to give it a break. If its energy is below 10%, it won't
want to eat or play. Wait a little while to let its energy come back. When it wants to do things, you
may feed it to satisfy its hunger and play with it to keep it happy. There is nothing more to this
because I had 9 hours to make this for an APCSP project. Enjoy!");
    setScreen("petScreen");
    startClock();
    updateStatsAndEmotions();
  }
  else
  {
    setProperty("startScreenInstructions", "text-color", rgb(255,0,0));
    setTimeout(function() {
      setProperty("startScreenInstructions", "text-color", rgb(0,0,0));
    }, 3000);
```

```
    }
  });

////////////////////////////////////////////////////
//*petScreen*
  onEvent("petScreenFeedButton", "click", function( ) {
    feedBlob();
  });

  onEvent("petScreenPlayButton", "click", function( ) {
    entertainBlob();
  });

  onEvent("petScreenCostumeButton", "click", function( ) {
    cycleCostumes();
    playSound("assets/Costume.mp3", false);
  });

  onEvent("petScreenRulesButton", "click", function( ) {
    setScreen("rulesScreen");
  });
////////////////////////////////////////////////////
//*rulesScreen*
  onEvent("rulesScreenBackButton", "click", function( ) {
      setScreen("petScreen");
    });
```